PROCESSOR DESIGN STUDIO

Complete Web Dashboard Manual, Feature Breakdown & Professor Demonstration Guide

Domain: Computer Architecture & Parallel Systems | **ISA:** 32-bit Custom RISC (19 Instructions, 16 Regs) | **Tech Stack:** C++17 (OpenMP), Node.js (Express), React (Vite) | **Live Ports:** Backend: localhost:3001 · Frontend: localhost:5173

1. System Architecture & Live Deployment Status

Processor Design Studio is a cycle-accurate hardware simulation and design-space exploration platform. It decouples the simulation kernel (C++17 for raw compute and OpenMP parallelism) from the backend API and React frontend.

Layer	Technology	Active Host Port / Path	Role & Execution Model
Sim Core	C++17, OpenMP, CMake	sim-core/build/sim.exe	Executes functional / 5-stage pipelined CPU simulation, L1 cache modeling, branch prediction, and multi-threaded parameter sweeps.
Backend API	Node.js, Express, child_process	http://localhost:3001	Thin wrapper invoking sim.exe via stdin/stdout JSON streaming. Exposes endpoints: /health, /benchmarks, /simulate, /sweep.
Frontend UI	React 18, Vite 5, Vanilla CSS	http://localhost:5173	Interactive three-tab web interface: Configuration Builder, Pipeline Visualizer, and Architectural Comparison Dashboard.

2. Webpage Tab 1: Configuration Builder (http://localhost:5173)

Purpose: Allows custom microarchitecture configuration, single cycle-accurate simulation triggers, and real-time inspection of 7 execution telemetry metrics and raw configuration JSON vectors.

Feature Component	How It Works Technically (C++ Core)	Architectural Purpose / Demonstration Goal
Benchmark Workload Selection	Queries GET /benchmarks to fetch available assembly files: loop_sum, factorial, bubble_sort, array_sum, hazard_heavy, tight_loop.	Allows testing distinct workloads: memory-intensive, branch-intensive, recursive call stacks, and hazard-heavy sequences.
5-Stage Pipelining Toggle	When ON, initializes PipelinedSimulator modeling IF/ID/EX/MEM/WB with latch registers. When OFF, switches to single-cycle Simulator (CPI = 1.000).	Demonstrates ideal single-cycle execution versus real-world pipelined instruction overlap and pipeline latency overheads.
Data Forwarding Toggle	Toggles EX/MEM & MEM/WB bypass multiplexers into ALU input operands. Detects RAW hazards: (id_ex.rs == ex_mem.rd) and forwards without stall.	Demonstrates hardware forwarding paths resolving Read-After-Write hazards without inserting bubble stalls into the execution stage.
Hazard Stalling Toggle	Detects Load-Use hazards where id_ex.opcode == LW and freezes PC + IF/ID latches while injecting a NOP bubble into ID/EX latch.	Shows mandatory pipeline bubbles when data cannot be forwarded in time (load-use dependency delay).
L1 Data Cache Hierarchy	Configurable Cache Size (0–128 KB, 0=disabled), Block Size (16–128 B), Associativity (1-way to 8-way), and Replacement (LRU, FIFO, Random).	Demonstrates spatial and temporal locality. Shows how set associativity eliminates conflict misses on memory-dense algorithms.
Branch Predictor Strategy	Selects branch prediction strategy: STATIC_NOT_TAKEN, STATIC_TAKEN, ONE_BIT, or TWO_BIT_SATURATING. Tracks branch hits and flushes.	Illustrates control hazard mitigation. Shows how 2-bit hysteresis counters prevent thrashing on loop termination branches.

Telemetry Output Metrics Explained:

- **Total Cycles:** Elapsed clock cycles from first fetch to HALT retirement. | • **Instructions Executed:** Useful architectural instructions committed in WB.
- **CPI (Cycles Per Instruction):** Total Cycles / Instructions Executed. Pipelined approaches 1.0. | • **IPC:** 1 / CPI (throughput).
- **Stall Cycles:** Cycles frozen due to RAW hazards or Load-Use bubbles. | • **Cache Hit Rate:** % of memory accesses resolved in L1 Cache.
- **Branch Misprediction Rate:** % of conditional branches where speculative fetch was wrong, incurring a 2-cycle pipeline flush.

3. Webpage Tab 2: Pipeline Stage Trace Visualizer

Purpose: Acts as an interactive logic analyzer. Provides cycle-by-cycle inspection of the 5 pipeline stage latches (IF, ID, EX, MEM, WB), visualizes instruction movement, and highlights data/control hazards.

Pipeline Stage	Active Hardware Component	Function in Toy RISC Architecture
IF (Fetch)	Program Counter & Instruction Memory	Fetches the 32-bit instruction word pointed to by PC from memory, increments PC by 4.
ID (Decode)	Instruction Decoder & Register File	Splits 32-bit word into Opcode/Funct/Reg indices. Reads source registers $rs and $rt from Register File ($r0–$r15).
EX (Execute)	ALU & Branch Resolution Unit	Performs arithmetic/logical operation or evaluates branch conditions (BEQ, BNE, BLT, BGE). Target address calculated.
MEM (Memory)	L1 Data Cache Controller	Performs word-aligned memory access for LW (read) and SW (write) against L1 Cache / flat memory.
WB (Write Back)	Register File Write Port	Commits ALU result or loaded memory value into destination register $rd or $rt ($r0 writes ignored).

Visualizer Controls & Hazard Highlights:

- **Cycle Step Buttons (< PREV / NEXT >):** Step forward/backward cycle-by-cycle to trace instruction flow across stages.
- **Hazard Badges:** Highlights DATA HAZARDS (RAW) in Amber, CONTROL HAZARDS (Branches) in Magenta, and STALL BUBBLES in Cyan.
- **Bypass Telemetry:** Badges confirm active EX/MEM -> EX and MEM/WB -> EX hardware forwarding lines.

4. Webpage Tab 3: Comparison & Architectural Sweep Dashboard

Purpose: Executes multi-dimensional parameter sweeps across architectural combinations, aggregates results via OpenMP multithreading, and generates live comparative charts and a searchable data matrix.

Dashboard Component	How It Works Technically	Analytical Purpose & Value
OpenMP Parallel Sweep Runner	Sends { parallel: true } to backend. C++ core executes #pragma omp parallel for schedule(dynamic,1) across CPU cores.	Accelerates architectural exploration by running 150+ configurations concurrently, capturing speedup and Amdahl's Law serial fraction.
Telemetry Summary Strip	Computes real-time statistics: Total Configs Evaluated, Optimum (Minimum) CPI, Maximum CPI, and Cross-Config Mean CPI.	Gives instant high-level visibility into best-performing and worst-performing microarchitectural designs.
Dynamic Bar Charts	Renders live comparative charts with selectable metric axis: CPI, IPC, Stall Cycles, Misprediction Rate, Cache Hit Rate, or Wall Time.	Visually correlates microarchitectural features (e.g. branch predictor algorithm vs misprediction rate).
Interactive Data Matrix	Sortable HTML table with column filters for Workload and Predictor. Displays CPI, IPC, Stalls, Hit %, Mispred %, and runtime.	Allows in-depth numerical inspection of any individual configuration tested during the sweep.

5. Step-by-Step Professor Demonstration Guide (5 Key Experiments)

Follow these exact steps during your demonstration to showcase the full depth of your microarchitecture platform. Each test case includes exact UI clicks and a word-for-word explanation script.

Test Case 1: Data Forwarding vs. RAW Hazard Stalling (Pipeline Unit)

Steps on UI: In **Config Builder**, select Benchmark: hazard_heavy.asm, turn Pipeline **ON**.
• **Exp A (No Forwarding):** Uncheck 'Data Forwarding', leave 'Hazard Stalling' checked. Click **RUN SIMULATION**. Observe CPI: **~2.384** (Stalls: 36–41).
• **Exp B (With Forwarding):** Check 'Data Forwarding' **ON**. Click **RUN SIMULATION**. CPI drops to **~1.307** (Stalls drop to 4–10).
• Click **'INSPECT PIPELINE TRACE'** to show EX/MEM bypass in action.

What to Say: *'Professor, here we observe our hazard detection and forwarding units. Without forwarding, consecutive Read-After-Write dependencies force the hazard unit to stall the pipeline, inserting bubbles into the EX stage and inflating CPI to 2.384. When forwarding is enabled, bypass paths route results directly from the EX/MEM and MEM/WB latches back to ALU inputs, boosting throughput by over 45%.'*

Test Case 2: Cache Associativity & Eviction Policies (Memory Unit)

Steps on UI: In **Config Builder**, select Benchmark: array_sum.asm or bubble_sort.asm.
• **Exp A (Direct-Mapped):** Cache Size: 1 KB, Block Size: 16 B, Associativity: **Direct Mapped**. Click **RUN SIMULATION**. Hit Rate: **~75.0%**.
• **Exp B (4-Way Associative LRU):** Cache Size: 4 KB, Block Size: 64 B, Associativity: **4-Way**, Policy: **LRU**. Click **RUN SIMULATION**. Hit Rate increases to **>90% - 93%**.

What to Say: *'With a 1KB direct-mapped cache, memory accesses suffer conflict misses whenever active array addresses hash to the same set index. Expanding to a 4KB 4-way set-associative cache with Least Recently Used (LRU) eviction eliminates conflict misses and preserves spatial block locality, driving hit rates above 90%.'*

Test Case 3: Branch Predictor Sophistication (Control Flow Unit)

Steps on UI: In **Config Builder**, select Benchmark: loop_sum.asm.
• **Exp A:** Set Predictor to STATIC_TAKEN. Click **RUN SIMULATION**. Misprediction Rate: **~90.9%** (10 flushes out of 11 branches).
• **Exp B:** Set Predictor to TWO_BIT_SATURATING. Click **RUN SIMULATION**. Misprediction Rate drops dramatically to **~9.09%** (only 1 flush on final loop exit).

What to Say: *'In loop structures, exit branches evaluate to False during iterations. Static Taken prediction incurs a 2-stage pipeline flush penalty on every iteration. Our 2-bit saturating counter predictor uses a 4-state hysteresis state machine (Strongly/Weakly Taken/Not-Taken) which absorbs iterations and achieves 90.9% accuracy.'*

Test Case 4: Functional Single-Cycle Baseline vs. Cycle-Accurate Pipelining

Steps on UI: In **Config Builder**, select Benchmark: factorial.asm.
• **Exp A:** Turn 5-Stage Pipelining **OFF**. Run simulation. Observe: **Total Cycles == Instructions Executed (Ideal CPI = 1.000)**.
• **Exp B:** Turn 5-Stage Pipelining **ON**. Run simulation. Observe: Total Cycles includes pipeline fill latency and subroutine call delays.

What to Say: *'This validates architectural fidelity. In functional mode, each instruction commits in a single cycle (ideal CPI 1.0). In pipelined mode, our engine models the real hardware cost: pipeline fill cycles, subroutine jumps (JAL/JR), and return address latching.'*

Test Case 5: OpenMP Parallel Exploration & Amdahl's Law Speedup

Steps on UI: Navigate to **Comparison Dashboard** tab. Toggle **OPENMP PARALLEL** mode. Click **RUN FULL PARAMETER SWEEP**. Watch live execution across 150+ configurations.

What to Say: *'To evaluate hundreds of design combinations, we parallelized the C++ simulation core using OpenMP (#pragma omp parallel for). As documented in our benchmark report, we measured an empirical speedup of 1.48x on 2 cores and derived the non-parallel serial fraction ($s \approx 35.1\%$) due to JSON parsing and benchmark file I/O per Amdahl's Law.'*

6. Professor Viva Defense: Frequently Asked Questions & Answers

Be prepared for these standard questions that professors and evaluators frequently ask during project defense:

Q1: How does your tool compare to gem5 or SimpleScalar?

gem5 is an industry-standard full-system simulator for production ISAs (x86, ARM, RISC-V) running millions of instructions with OS support, but it is 10^5x slower than silicon and lacks interactive visual pipeline inspectors. Our Processor Design Studio focuses on rapid microarchitectural design-space exploration (DSE) of fundamental textbook concepts (Hennessy & Patterson) using a custom 19-instruction toy RISC ISA and native OpenMP multi-core parallel sweeps.

Q2: Why is register $r0 hardwired to zero?

Hardwiring $r0 to 0 simplifies the ISA. It allows synthesis of NOPs (ADD $r0, $r0, $r0), register clearing, unconditional register copying (ADD $rd, $rs, $r0), and absolute addressing offsets without requiring dedicated ALU instructions.

Q3: Why can't all RAW hazards be eliminated by data forwarding?

Load-Use dependencies cannot be completely forwarded because loaded data is not available until the end of the MEM stage, whereas the dependent instruction needs the operand at the beginning of the EX stage. This requires a mandatory 1-cycle stall bubble inserted by the hazard unit.

Q4: What is the write policy of your cache?

Our L1 Data Cache implements Write-Through with No-Write-Allocate. On a write hit, both the cache line and backing main memory are updated. On a write miss, the write updates memory directly without fetching the block into cache, avoiding cache pollution on single-use writes.

Q5: Why did OpenMP speedup plateau at higher thread counts in your tests?

Because each individual benchmark execution takes less than 1 millisecond, the serial overhead (parsing stdin JSON, loading .asm files from disk, and stringifying JSON results) accounts for approximately 35% of the total wall time. According to Amdahl's Law, with a serial fraction $s \approx 35\%$, maximum theoretical speedup is capped at $1/s \approx 2.85$x.

Automated Verification Summary: 253 / 253 automated unit and integration tests passing in sim-core/build/test_all.exe across 16 microarchitectural groups.